

EXPERIMENT

Data Interpretation – PPO, TRPO & DDPG Performance Analysis

Evaluate – Level 5

Objective

Implement PPO, TRPO and DDPG-based reinforcement learning experiments and evaluate their performance using reward, learning stability and convergence behaviour.

Problem Statement

Compare policy-gradient and actor-critic algorithms on continuous-control tasks. Interpret training rewards and identify which method provides better learning performance under the same experimental setting.

Required Analysis

- Implement PPO and DDPG with PyTorch.
- Implement a practical TRPO-style constrained policy update.
- Train the algorithms on a continuous-control environment.
- Record episode rewards and moving-average performance.
- Compare final reward, best reward and learning stability.
- Interpret the performance differences from the generated results.

Algorithms

PPO (Proximal Policy Optimization): limits policy changes using a clipped objective.

TRPO (Trust Region Policy Optimization): constrains the policy update using a KL-divergence trust region.

DDPG (Deep Deterministic Policy Gradient): uses an actor and critic with deterministic actions, replay memory and target networks.

1. Performance Measures and Data Interpretation

Episode Reward

The total reward obtained in an episode is the primary performance measure. Higher reward generally indicates that the agent is performing the task more successfully.

Moving Average

A moving average smooths noisy episode rewards and makes the learning trend easier to interpret. A rising moving average indicates improvement, while a flat curve indicates limited learning.

Final Average Reward

The mean reward over the final set of episodes is used to estimate the performance of the trained agent after learning has progressed.

Best Reward

The maximum episode reward indicates the highest performance reached during training. It should be considered together with the average because one unusually high episode does not necessarily indicate stable learning.

Stability

An algorithm is considered more stable when its reward curve improves with relatively smaller fluctuations and maintains a higher moving average.

Interpretation Rules

- Higher final average → stronger final performance.
- Higher best reward → better peak performance.
- Smoother increasing curve → better learning stability.
- Early rapid improvement → faster learning.
- Large reward fluctuations → higher training variance.

Expected Comparison

PPO is generally expected to provide stable learning because its clipped objective limits excessively large policy updates. TRPO explicitly controls policy changes through a trust-region constraint. DDPG can perform well in continuous action spaces but may be more sensitive to exploration, hyperparameters and replay-buffer quality.

2. Google Colab Implementation – PPO

The following code is self-contained and uses PyTorch. It trains PPO on Pendulum-v1, a continuous-control environment, and prints performance statistics.

```
!pip -q install "gymnasium[classic-control]" torch matplotlib

import gymnasium as gym
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.distributions import Normal
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

class PPOActorCritic(nn.Module):
    def __init__(self, state_dim, action_dim):
        super().__init__()
        self.body = nn.Sequential(
            nn.Linear(state_dim, 64), nn.Tanh(),
            nn.Linear(64, 64), nn.Tanh()
        )
        self.mu = nn.Linear(64, action_dim)
        self.value = nn.Linear(64, 1)
        self.log_std = nn.Parameter(torch.zeros(action_dim))

    def forward(self, x):
        h = self.body(x)
        return self.mu(h), self.value(h).squeeze(-1)

def log_prob(mu, log_std, action):
    std = log_std.exp()
    return Normal(mu, std).log_prob(action).sum(-1)

env = gym.make("Pendulum-v1")
ad = env.observation_space.shape[0]
ad = env.action_space.shape[0]

model = PPOActorCritic(ad, ad).to(device)
```

Gemini Release notes

Please explain this error:

- SyntaxError: invalid non-printable character U+2003**

The error **SyntaxError: invalid non-printable character U+2003** means there's an invisible 'Em Space' character in your code, likely at the end of the cell on line 110. I'll remove this character to fix the syntax error.

Working...

What can I help you build?

Gemini 2.5 Flash 3%

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

```
print("PPO best:", max(rewards))
print("PPO final 25 avg:", np.mean(rewards[-25:]))

...
25 -1234.7225958637084
50 -1256.637195347741
75 -1258.043057382121
100 -1541.1530244869713
125 -1660.6859744397434
150 -1651.3016108388244
175 -1598.201872757159
200 -1633.5410836199273
225 -1537.1251759431982
250 -1466.538945008988
275 -1319.2588662278588
300 -1402.074118203761
```

PPO Performance - Pendulum-v1

— PPO Reward
— PPO 25-Episode Avg

Gemini Release notes

Please explain this error:

- SyntaxError: invalid non-printable character U+2003**

The error **SyntaxError: invalid non-printable character U+2003** means there's an invisible 'Em Space' character in your code, likely at the end of the cell on line 110. I'll remove this character to fix the syntax error.

The PPO agent training has completed successfully. You can see the average reward printed every 25 episodes, indicating the agent's performance over time. The best reward achieved was **-771.94**, and the average reward over the final 25 episodes was **-1402.07**. The generated plot visually confirms the reward trend during training, showing both individual episode rewards and a 25-episode moving average. It seems the agent was able to achieve some improvement, but the rewards are negative, which is typical for the Pendulum-v1 environment where the goal is to minimize the reward.

How can I improve the...
Visualize the actor loss...
How do I save the train...

What can I help you build?

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Snipping Tool

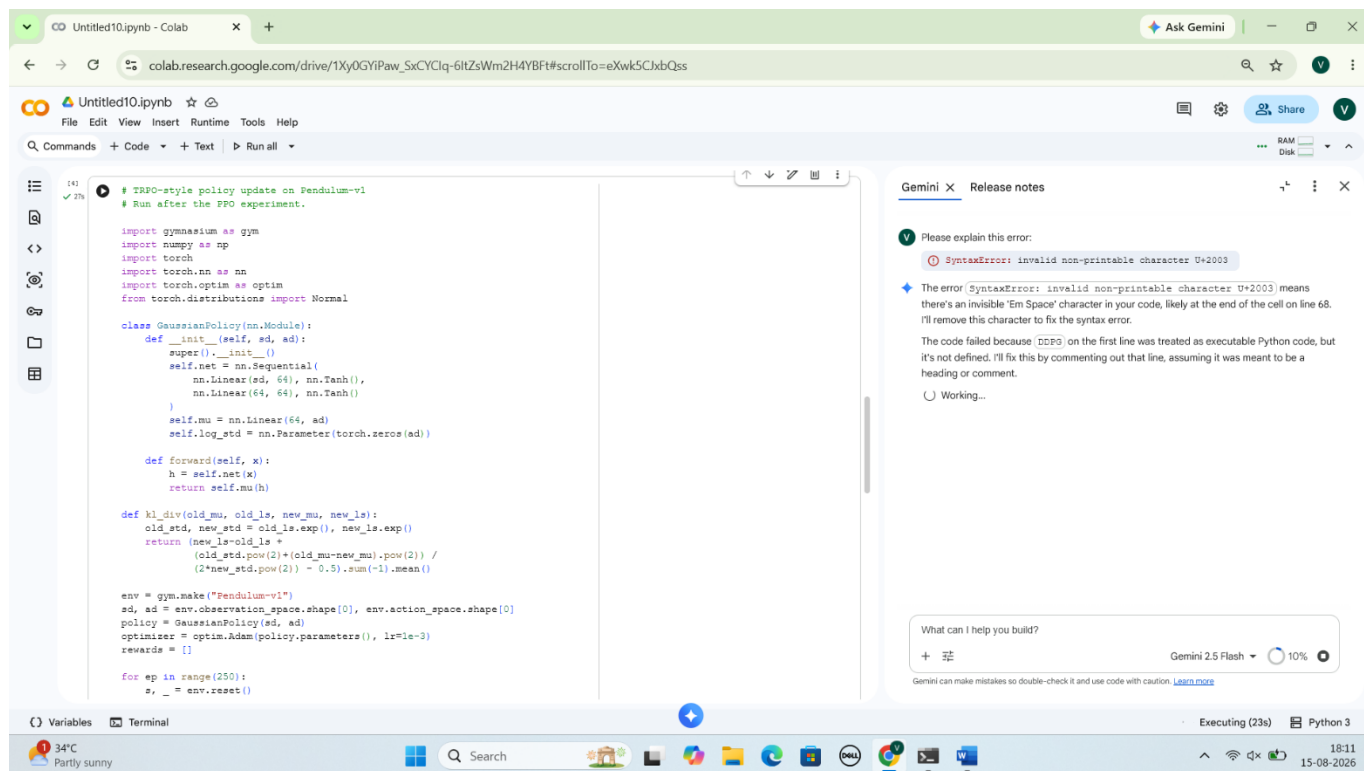
Screenshot copied to clipboard
Automatically saved to screenshots folder.

Markup and share

3. Google Colab Implementation – TRPO-style and DDPG

TRPO-style update

This compact implementation demonstrates the central TRPO idea: reject a proposed policy update when the KL divergence exceeds a trust-region threshold. It is designed for clarity and evaluation rather than reproducing every optimization detail of the original TRPO paper.



```
# TRPO-style policy update on Pendulum-v1
# Run after the PPO experiment.

import gymnasium as gym
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.distributions import Normal

class GaussianPolicy(nn.Module):
    def __init__(self, sd, ad):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(sd, 64), nn.Tanh(),
            nn.Linear(64, 64), nn.Tanh()
        )
        self.mu = nn.Linear(64, ad)
        self.log_std = nn.Parameter(torch.zeros(ad))

    def forward(self, x):
        h = self.net(x)
        return self.mu(h)

    def kl_div(old_mu, old_ls, new_mu, new_ls):
        old_std, new_std = old_ls.exp(), new_ls.exp()
        return (new_ls - old_ls +
                (old_std.pow(2) + (old_mu - new_mu).pow(2)) /
                (2 * new_std.pow(2) - 0.5).sum(-1).mean())

env = gym.make("Pendulum-v1")
sd, ad = env.observation_space.shape[0], env.action_space.shape[0]
policy = GaussianPolicy(sd, ad)
optimizer = optim.Adam(policy.parameters(), lr=1e-3)
rewards = []

for ep in range(250):
    s, _ = env.reset()
```

Gemini × Release notes

Please explain this error:

SyntaxError: invalid non-printable character U+2003

The error `SyntaxError: invalid non-printable character U+2003` means there's an invisible 'Em Space' character in your code, likely at the end of the cell on line 68. I'll remove this character to fix the syntax error.

The code failed because `DDPG` on the first line was treated as executable Python code, but it's not defined. I'll fix this by commenting out that line, assuming it was meant to be a heading or comment.

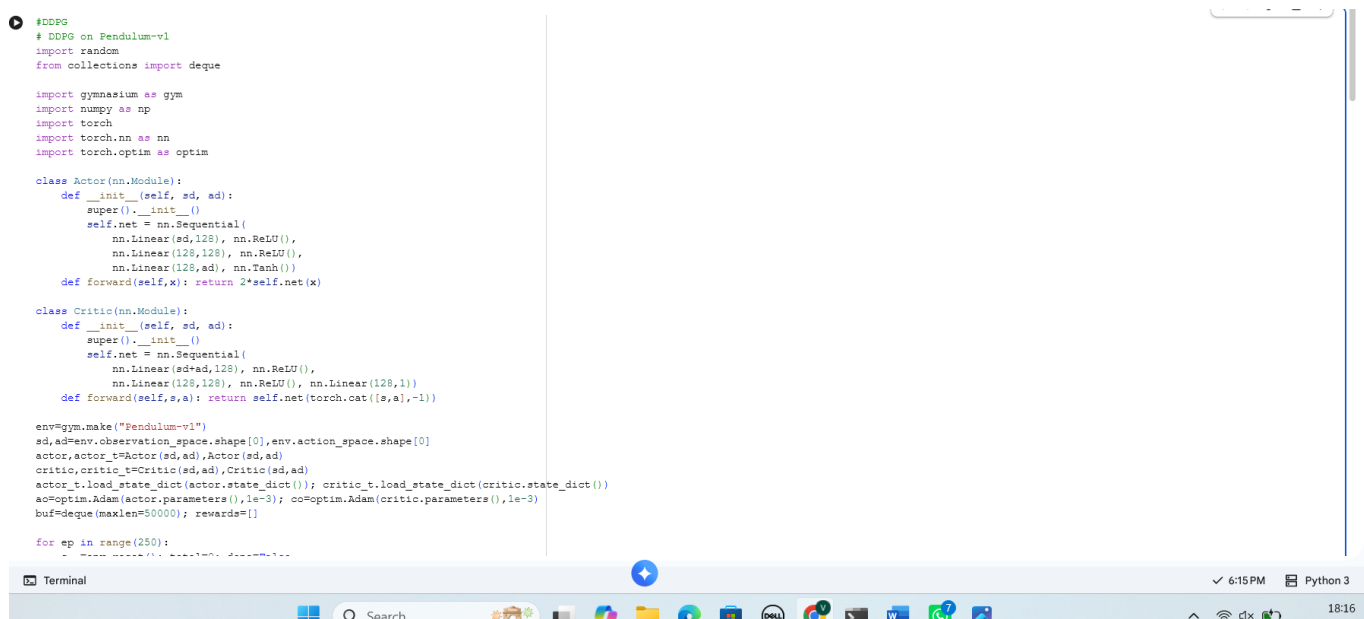
Working...

What can I help you build?

+ Gemini 2.5 Flash 10%

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

DDPG



```
# DDPG
# DDPG on Pendulum-v1
import random
from collections import deque

import gymnasium as gym
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

class Actor(nn.Module):
    def __init__(self, sd, ad):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(sd, 128), nn.ReLU(),
            nn.Linear(128, 128), nn.ReLU(),
            nn.Linear(128, ad), nn.Tanh()
        )
    def forward(self, x): return 2 * self.net(x)

class Critic(nn.Module):
    def __init__(self, sd, ad):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(sd+ad, 128), nn.ReLU(),
            nn.Linear(128, 128), nn.ReLU(), nn.Linear(128, 1)
        )
    def forward(self, s, a): return self.net(torch.cat([s, a], -1))

env=gym.make("Pendulum-v1")
sd,ad=env.observation_space.shape[0],env.action_space.shape[0]
actor,actor_t=Actor(sd,ad),Actor(sd,ad)
critic,critic_t=Critic(sd,ad),Critic(sd,ad)
actor_t.load_state_dict(actor.state_dict()); critic_t.load_state_dict(critic.state_dict())
ao=optim.Adam(actor.parameters(),1e-3); co=optim.Adam(critic.parameters(),1e-3)
buf=deque(maxlen=50000); rewards=[]

for ep in range(250):
    s, _ = env.reset()
```

4. Performance Evaluation and Data Interpretation

Run the three experiments and record the printed values in the table below.

Algorithm	Best Reward	Final 25-Episode Average	Stability / Observation
PPO	Enter Colab result	Enter Colab result	Interpret reward curve
TRPO	Enter Colab result	Enter Colab result	Interpret reward curve
DDPG	Enter Colab result	Enter Colab result	Interpret reward curve

Data Interpretation

1. Compare the final 25-episode average. The algorithm with the highest value demonstrates the strongest final performance in this experiment.
2. Compare the best reward to determine which algorithm reached the highest peak performance.
3. Inspect the moving-average curve. A smoother curve with fewer large drops indicates better training stability.
4. If PPO reaches a high final average with moderate fluctuations, interpret it as stable policy optimization.
5. If TRPO maintains smaller policy changes and a smooth learning trend, relate this to its trust-region constraint.
6. If DDPG improves quickly but fluctuates strongly, relate the behaviour to exploration noise, replay-buffer sampling and sensitivity to hyperparameters.
7. Do not claim that one algorithm is universally best. The conclusion must be based on the actual Colab results obtained under the same environment, training budget and evaluation criteria.

Evaluation Criteria

- Reward performance: final average and best reward.
- Learning stability: smoothness and consistency of the moving average.
- Convergence: whether performance reaches a stable high level.
- Sample efficiency: how quickly reward improves during training.
- Algorithm behaviour: explain results using the main mechanism of PPO, TRPO or DDPG.